Bioinformatics, 36(16), 2020, 4508–4509
doi: 10.1093/bioinformatics/btaa558
Advance Access Publication Date: 9 July 2020
Applications Note

Sequence analysis

DNA Chisel, a versatile sequence optimizer

Valentin Zulkower * and Susan Rosser

Edinburgh Genome Foundry, SynthSys, School of Biological Sciences, University of Edinburgh, Edinburgh EH93BF, UK

*To whom correspondence should be addressed.

Associate Editor: Yann Ponty

Received on December 18, 2019; revised on February 26, 2020; editorial decision on May 3, 2020; accepted on July 1, 2020

Abstract

Motivation: Accounting for biological and practical requirements in DNA sequence design often results in challenging optimization problems. Current software solutions are problem-specific and hard to combine.

Results: DNA Chisel is an easy-to-use, easy-to-extend sequence optimization framework allowing to freely define and combine optimization specifications via Python scripts or Genbank annotations.

Availability and implementation: The framework is available as a web application (https://cuba.genomefoundry.org/sculpt_a_sequence) or open-source Python library (see at https://github.com/Edinburgh-Genome-Foundry/DNAChisel for code and documentation).

Contact: valentin.zulkower@gmail

Supplementary information: Supplementary data are available at *Bioinformatics* online.

1 Introduction

Advances in DNA synthesis have made it possible for biologists to routinely order DNA constructs with custom nucleotide sequences (Kosuri and Church, 2014). While a sequence's primary purpose may be the study or engineering of an organism, its design may also account for manufacturability, host compatibility and other practical requirements, resulting in complex multi-constrained optimization problems.

Software solutions have been proposed to address various scenarios, including host-specific codon optimization or harmonization (Claassens *et al.*, 2017; Richardson *et al.*, 2012), gene expression enhancement via CpG island enrichment (Raab *et al.*, 2010), the design of biologically neutral sequences (Casini *et al.*, 2014) or the removal of synthesis-impeding DNA patterns (Oberortner *et al.*, 2017). However, these projects focus on specific objectives and predetermined sequence locations (such as coding regions), and are hard to integrate in the same workflow, as their optimizations may undo one another. The D-tailor framework (Guimaraes *et al.*, 2014) proposed a programmatic solution to this problem, enabling users to freely define and combine specifications via Python scripts, with a focus on the exploration of multi-objective problems.

DNA Chisel builds on these approaches with a focus on usability and versatility. It can be used as Python library, a command-line interface, or a web application, allowing researchers to model constraints and optimization objectives via either Python scripts or Genbank annotations. Over 15 built-in specification classes (listed in Supplementary Section S1A) can be composed to handle any combination of the optimization objectives mentioned above, and extended with user-defined specifications. Finally, the framework introduces new heuristics for efficient local sequence optimization, and produces comprehensive optimization reports for traceability and troubleshooting.

2 Usage

An optimization problem is defined in DNA Chisel by a list of global or local specifications against which a starting linear or circular sequence will be optimized. A specification can be either a hard constraint, which must be satisfied in the final sequence, or an optimization objective, whose score must be maximized. For instance, specification AvoidChanges can be used as a constraint to forbid sequence modifications in a given region, or as an objective to simply penalize changes in that region. In presence of multiple optimization objectives, which can be attributed relative weights, DNA Chisel will seek to maximize the total weighted score using the heuristics described in the next section (an example of multi-objective gene optimization is provided in Supplementary Section S1B).

A problem can be defined by annotating a Genbank record to indicate the nature and scope of the different specifications to be applied to the record's sequence, as shown in Figure 1A. This can be done interactively using a free Genbank editing software such as SnapGene Viewer (www.snapgene.com) or Benchling (www.benchling.com). Annotated Genbank files can be uploaded to a public web application (Fig. 1B), which will optimize the sequence and return a multi-file report (see Supplementary Information S2 for an example). The report features the optimized sequence in Genbank format with annotations indicating modified nucleotides, a PDF report summarizing the changes (as shown in Fig. 1C), and in case of unsuccessful constraint resolution, a troubleshooting figure representing the problematic region.

© The Author(s) 2020. Published by Oxford University Press. All rights reserved. For permissions, please e-mail: journals.permissions@oup.com

Downloaded from https://academic.oup.com/bioinformatics/article/36/16/4508/5869515 by Cold Spring Harbor user on 13 December 2025

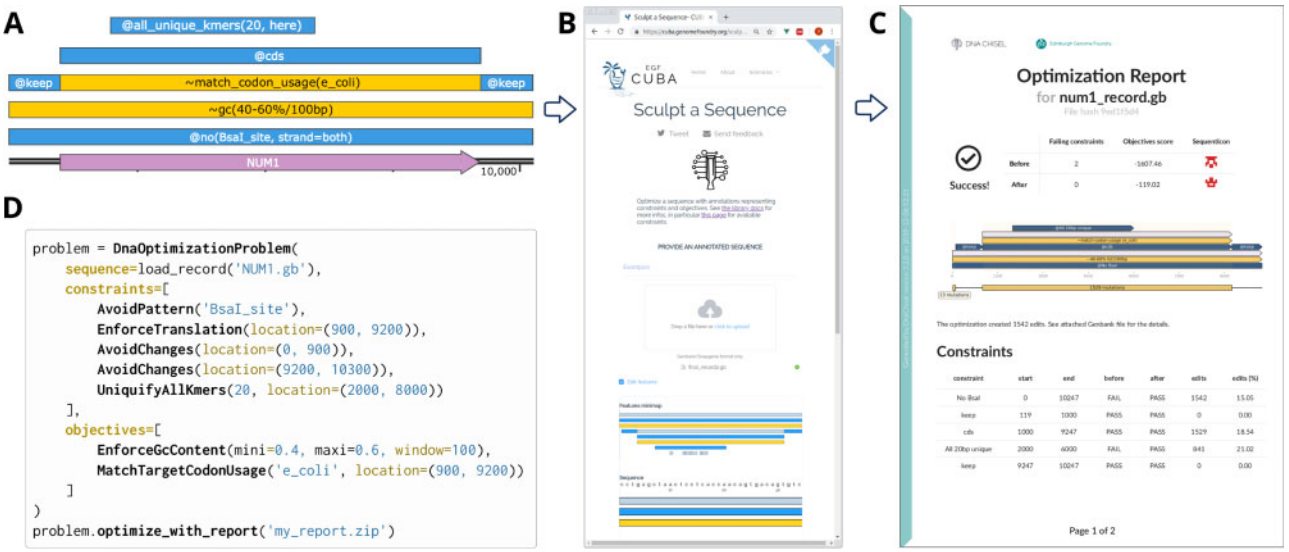

Fig. 1. Using DNA Chisel online and via scripts. (**A**) Sequence optimization problem defined via Genbank annotations. Prefix '~' in annotations indicate optimization objectives, while '@' denotes hard constraints. Gene NUM1 of *Saccharomyces cerevisiae* will be codon optimized for *Escherichia coli*, and have a GC content of 40–60% on every 100-nucleotide window. Flanking promoter and terminator regions are protected against changes by @keep annotations (shorthand for @AvoidChanges). BsaI restriction sites will be removed [@no(BsaI_site) constraint]. Undesirable homologies occurring naturally in the gene sequence will be removed in the region marked @all_unique_kmers, using codon-synonymous mutations only, as enforced by the @cds constraint. (**B**) Screenshot of the online DNA Chisel application featuring a file dropzone and a basic annotations editor. (**C**) PDF report returned by DNA Chisel to summarize the problem's specifications and highlight constraint resolutions and objective score improvements. (**D**) Python script defining the same optimization problem as in panel A

Optimization problems can also be defined and resolved via Python scripts (Fig. 1D), enabling the programmatic definition of complex optimization scenarios involving hundreds of specifications. Supplementary Section S1C and D provide example scripts for mass sequence pattern removal, and genome-scale gene domestication.

3 Implementation

DNA Chisel is implemented in Python. It relies on the Biopython library (Cock *et al.*, 2009) for Genbank operations and restriction enzyme data, and on the Codon Usage Database (Nakamura, 2000) for codon usage data.

The optimization algorithm, described in more detail in Supplementary Section S1E, has two main steps: (i) resolution of all hard constraints, ignoring optimization objectives, and (ii) objectives maximization with respect to the constraints. This separation of steps allows to detect incompatible constraints early, and quickly reaches satisfactory solutions.

During each step, unsatisfactory sequence regions are detected and separately optimized. The solver speeds up region optimization by creating a local version of the problem with simplified specifications locally equivalent to the original problem's specifications. A region's sequence is optimized via either a stochastic search (using random mutations of the sequence) or an exhaustive search through all possible sequence variants, depending on the number of variants. Specifications can also implement custom local resolution methods to improve the solver's efficiency. For instance, an insert(CGTCTC) constraint will attempt to place the pattern 'CGTCTC' at different locations of the sequence, rather than relying on random or exhaustive search to create the pattern in the sequence.

4 Framework extension with new specifications

The emergence of new biological applications often comes with new DNA sequence optimization requirements. In this perspective, DNA Chisel allows to define any new sequence specification that the Python language can express, without modifying the library's code or compromising the solver's efficiency. A new specification is defined by creating a new Python class with custom sequence evaluation and local resolution methods, and the class can optionally be registered with DNA Chisel's Genbank record parser to enable its use via Genbank annotations.

Acknowledgements

The authors thank Ye Chang, Paulina Kanigowska, Simone Pignotti and Li Xing for useful discussions on this project.

Funding

This work was supported by the Biotechnology and Biological Sciences Research Council [BB/M025659/1, BB/M025640/1 and BB/M00029X/1 to S.R.] and the BBSRC/MRC/EPSRC funded UK Centre for Mammalian Synthetic Biology [BB/M0101804/1 to S.R.] as part of the Research Council UK's Synthetic Biology for Growth programme.

Conflict of Interest: none declared.

References

Casini,A. *et al.* (2014) R2oDNA designer: computational design of biologically neutral synthetic DNA sequences. *ACS Synth. Biol.*, **3**, 525–528.

Claassens,N.J. *et al.* (2017) Improving heterologous membrane protein production in *Escherichia coli* by combining transcriptional tuning and codon usage algorithms. *PLoS One*, **12**, e0184355.

Cock,P.J.A. *et al.* (2009) Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, **25**, 1422–1423.

Guimaraes,J.C. *et al.* (2014) D-Tailor: automated analysis and design of DNA sequences. *Bioinformatics*, **30**, 1087–1094.

Kosuri,S. and Church,G.M. (2014) Large-scale de novo DNA synthesis: technologies and applications. *Nat. Methods*, **11**, 499–507.

Nakamura,Y. (2000) Codon usage tabulated from international DNA sequence databases: status for the year 2000. *Nucleic Acids Res.*, **28**, 292.

Oberortner,E. *et al.* (2017) Streamlining the design-to-build transition with build-optimization software tools. *ACS Synth. Biol.*, **6**, 485–496.

Raab,D. *et al.* (2010) The GeneOptimizer Algorithm: using a sliding window approach to cope with the vast sequence space in multiparameter DNA sequence optimization. *Syst. Synth. Biol.*, **4**, 215–225.

Richardson,S.M. *et al.* (2012) Design-a-gene with genedesign. *Methods Mol. Biol.*, **852**, 235–247.

Downloaded from https://academic.oup.com/bioinformatics/article/36/16/4508/5869515 by Cold Spring Harbor user on 13 December 2025